A Protocol for AI-Assisted Reproducibility Auditing: Specification and Empirical Agenda

Ghina Abdoul-Baki, Juan P. Aparicio, Bruno Barbarioli, Abel Brodeur*

Institute for Replication, Department of Economics, University of Ottawa

* Corresponding author: Abel Brodeur, abrodeur@uottawa.ca

Abstract

This paper specifies a protocol for running reproducibility audits with AI assistance under an accountable human coordinator, written so that the comments it produces can cite the procedure they followed. The protocol has five stages. The package is copied read-only. The coordinator writes a shared intake brief and declares it fallible; auditors are instructed to correct it. Five auditors run in parallel, one question each, blind to one another's work. Disagreements are settled by re-running the disputed computation, never by preferring an auditor, and a ledger dispositions every finding as included, merged, or excluded with a reason. Adversarial critique rounds attack the audit rather than the paper. Release is gated on twenty-seven checks, eight of which test the detectors themselves. A finished audit then hands its verifier a claims script — every finding numbered, severity-labelled, and paired with the code that checks it, in the deposited package's own language — so that a human can confirm or refute each claim and the verdicts can be counted. One principle governs the design: a detector that returns zero has established nothing until it has fired on a known positive, with its scope stated in advance. We pre-register seventeen quantities a later paper will estimate, headed by protocol compliance and protocol reproducibility.

Keywords: reproducibility; replication; research transparency; audit protocol; error detection

1. Introduction

A reproducibility audit asks whether a published result follows from the materials deposited with it. Done arithmetically, the job is self-verifying: statcheck recomputes a p-value from the test statistic printed beside it, and anyone can check the verdict by hand. An audit of the deposited package is not like that. Someone has to decide which numbers count as headline numbers, whether a gap between code and prose is a defect or a convention, whether a benign explanation has been ruled out. Each decision is a judgment, and a judgment leaves nothing a later reader can check unless the procedure that produced it was written down.

Such audits are increasingly machine-assisted, and the arithmetic of the field says they will stay that way. The volume needing audit grows about four percent a year (Bornmann et al., 2021) against a fixed supply of expert time (Aczel et al., 2021), and availability policies change what can be audited, not how much (Vilhuber, 2020, 2023). Yet the procedure these audits follow has never been written down in a form a comment can cite.

Two questions follow. What procedure should such an audit run, so that false positives are caught where they are created rather than after publication? And how should it be specified, so that the comments it produces can cite it? This paper answers both. It specifies a five-stage protocol for

AI-assisted auditing under an accountable human coordinator, states the principle that carries its error discipline, and pre-registers the quantities that will show whether the protocol works.

Three bodies of work set the terms, and none supplies the procedure. The consistency checkers, from GRIM to statcheck to the Carlisle method, show that targeted error detection is cheap and effective (Nuijten and Wicherts, 2024); but each is single-purpose and deterministic, and each reads the printed article, so its procedure is whatever it computes. The reproduction literature supplies rates rather than procedures (Chang and Li, 2022; Herbert et al., 2024; Brodeur et al., 2026a). The work on language models in review roles adds a constraint. Model feedback overlaps with human reviewers about as much as two humans overlap with each other (Liang et al., 2024). Models cannot judge research quality unaided (Thelwall, 2024). Agents complete a minority of the harder reproducibility tasks (Siegel et al., 2025). And AI-assisted teams outperform AI-led teams, but not human-only teams (Brodeur et al., 2026b). Supervision is required; determinism is unavailable as a source of error control; what remains is procedure.

Three contributions follow. The first is the specification (Section 3): five stages, fixed counting units, four rules binding every auditor, a ledger that dispositions every finding, and a fixed artifact set, written so that a comment can cite the checklist item it applied and the gate set under which it ran. A deterministic checker gets its error control for free. A judgment-laden audit has to buy it, and the price is writing the procedure down.

The second is the principle that takes over from determinism as the source of error control: a detector that returns zero has established nothing until it has fired on a known positive, with its scope stated in advance on five axes. Most checks in a good audit come back clean, and a clean result is the easiest thing in the world to produce by accident — a pattern that cannot match its target, a scan pointed at the wrong folder, a gate that fails without blocking anything. The principle is built into the artifact: eight of the twenty-seven release checks are self-tests of the detectors they gate. Section 4 turns the rule on this specification itself: the previous gate set shipped with two failure mechanisms live in its own gates, and this one records what its successor repaired and what it merely made visible.

The third is the empirical agenda (Section 5): seventeen quantities given operational definitions and units of observation before any cohort is drawn, so the paper that measures the protocol is pre-committed rather than a search over outcomes. Two are headline quantities a literature of reproduction rates cannot report. Protocol compliance, because pooling audits that did not follow the specification measures something else. And protocol reproducibility, because findings that do not recover under independent re-execution are evidence about the protocol, not about the paper.

Section 2 separates audit from replication. Section 3 is the specification proper — mapped in Figure 1, measured in Figure 2, and handed off in Figure 3 — closing on the false-clean rule. Section 4 covers the artifacts, the gates, and two defects live in them. Section 5 pre-registers the agenda; Sections 6 and 7 give scope and close.

2. Background

By *audit* we mean recomputing a paper's published statistics from the package its authors deposited: same data, same code, same specifications, run by someone other than the authors. That is not replication. Replication asks whether a finding recurs in new data, and it is the subject of the coordinated replication literature, from the Open Science Collaboration (2015) to Camerer et al. (2016).

The narrower question has a less encouraging record. Reproduction without author assistance succeeds in a minority of attempts even under mandatory deposit (Chang and Li, 2022; Hardwicke et al., 2018; Herbert et al., 2024), with higher rates in leading journals under the largest coordinated effort to date (Brodeur et al., 2026a).

Package-level audit differs from replication in what is inspected: the object is an artifact, not an effect. A paper can be internally consistent in every printed number while its sample-construction code drops the wrong factor level. A package can reproduce to the last decimal and still support a conclusion the analysis does not license. Audit therefore inspects raw files, construction code, estimation code, documentation, license, and disclosure risk, and it reads code against prose.

Not all error detection requires the package. A separate tradition builds cheap, targeted consistency tests that run on the published text alone: GRIM on whether a reported mean of integer-valued data is attainable at the stated sample size (Brown and Heathers, 2017), GRIMMER on measures of variability (Anaya, 2016), SPRITE on candidate distributions consistent with reported descriptives (Heathers et al., 2018), statcheck on p-values recomputed from reported test statistics (Nuijten et al., 2016; Nuijten and Wicherts, 2024), and the Carlisle method on trial baseline tables inconsistent with randomization (Carlisle, 2017).

These are the closest analogues, and the differences are structural. Each is single-purpose and deterministic, so essentially free of judgment-driven false positives. Each reads the printed article rather than the deposited package, so none reaches an internally consistent paper whose code is wrong. A protocol that reads code and prose inherits neither property and must buy error control procedurally.

Buying it procedurally is the only option left, given the mixed record of language models in review roles (Section 1) and the characteristic errors of automated pipelines (Kapoor and Narayanan, 2023). The Institute for Replication (2026) project extends the expertise question; its record could not be independently verified, and no claim here rests on it.

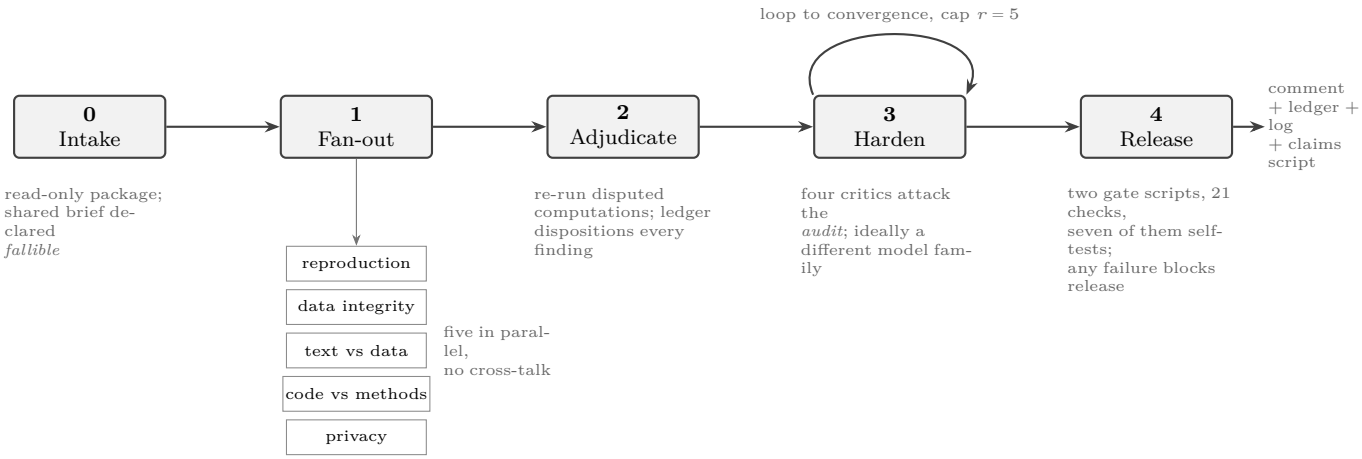
The machinery for buying it is standard software-quality practice — parallel independent reviewers, an adjudicator who re-derives rather than arbitrates, a decision log, self-testing checks, gated release — adapted to a setting where a false finding is not caught downstream by a failing build, and so has to be caught where it is made. Appendix C carries the failure modes that practice knows, in the terms an audit needs.

3. The Protocol

The protocol counts in fixed units, because Section 5 depends on them. A *claim string* is one phrasing in the retired-claims file; several can retire one claim. A *ledger row* is one finding from one auditor report, with its disposition. A *verdict* is one critic judgment on one draft item. A *defect* is a substantive error — in the audited paper, a *paper-defect*; in the comment, a *comment-defect*. These units are never summed across. A *headline quantity* is a number on which an abstract claim, a numbered conclusion, or the final discussion paragraph rests. The coordinator fixes that list in writing before reproduction begins, so recovery is judged against a list fixed ex ante rather than one chosen after the results are known.

The procedure runs in five stages, summarized in Figure 1 and specified below in the order they execute.

Figure 1. The five stages. Each audit ships a comment in I4R format with its ledger, its adjudi-



cation log, and a numbered claims script for the human verifier. Eight of the twenty-seven release checks are self-tests.

Stage 0 — Intake: a copy the audit cannot damage, and a brief that calls itself wrong

Intake begins by removing the audit's ability to contaminate its own evidence: paper and package go into a directory nothing in the audit may write to. The coordinator then writes one shared brief — what the paper claims, what the deposit contains, which files build the analysis sample, and where reproduction will be hard.

The brief is declared fallible, in three places. The coordinator is told not to trust it. Auditors are told it may contain errors and that saying so is part of the job. Critics may not manufacture results for materials the brief describes but the deposit lacks. Corrections go to a separate artifact and count as findings. The redundancy is not decoration: the brief is the only document all five auditors read, so an error in it can reach the comment wearing five-way apparent agreement.

Stage 1 — Fan-out: five auditors, one question each, no view of each other

Each auditor gets one question and cannot see the others' work: recovery of the published numbers from the raw data; data integrity, covering duplicates, impossible values, and fabrication signatures; prose against tables against figures; code against the Methods section; and privacy and archive hygiene. Appendix A gives the checklist, cited by item number in comments.

Four rules bind every auditor. Every claim carries a label — CONFIRMED, LIKELY, or SPECULATIVE. Negative results are mandated, and they are reported as tests of the paper, never as corrections of a draft. Every finding follows a fixed template: what, located to file, sheet, cell, or line; evidence, meaning a script that reproduces it; the benign explanations ruled out; the impact on published conclusions, written as *none* where there is none; and a remedy. And motive is never attributed.

Which model runs which auditor. The five questions are not equally hard, and the protocol does not require one model behind all of them. A controlled comparison on one paper — five Sonnet 5 and five Opus 5 auditors over byte-identical prompts, same deposit, same coordinator, arms blind to each other, with the scoring key written before either arm's reports were read — found the difference concentrated rather than diffuse. Both arms located the same defects, and the discriminator predicted in advance did not discriminate: both gave the correct answer on

whether the deposited inference was paired, and both placed the exceptions more precisely than the coordinator's own ground truth had. The separation appeared instead at the step from defect to cause. On the audit's central finding, the weaker arm localised the anomaly to the correct condition and column with a well-evidenced dispersion statistic; the stronger arm rebuilt the paper's undocumented derivation of its outcome, identified that part of one condition had been computed from a wrong input, and showed that correcting it changed the exception the abstract states. Four of the run's conclusion-changing findings came from the stronger arm alone, all from the reproduction, data-integrity and code areas. The pattern generalised across the five areas as *localise* against *explain*.

Two operational consequences follow. Text-against-tables is enumeration against a fixed corpus and tolerates the cheaper model; reproduction, data integrity and code do not, because that is where causes live. Privacy and archive looks like the cheapest area and is not: its real function is catching errors in the intake brief, and in this comparison the weaker arm wrote "Corrections to the brief: None" while naming, elsewhere in its own report, the fact that contradicted the brief. The brief in question carried a first author the coordinator had invented. Model identity per role per round is already recorded under Q15; Q18 makes the sensitivity itself a measured quantity rather than a standing assumption.

Stage 2 — Adjudication: re-run the computation, never pick a winner

Disagreements between auditors are settled by re-running the disputed computation, not by judging which auditor is more credible: a tested negative beats an untested assumption. Superseded decisions are marked WITHDRAWN with a reason rather than deleted.

Every finding from all five reports then enters the ledger as one row: INCLUDED, MERGED into a named section, or EXCLUDED with a reason a critic would accept. "No numerical impact" is not an acceptable reason, because most findings in a good audit have none. The ledger and the adjudication log ship with the audit materials and never appear in the comment itself — process narration invites authors to litigate the audit rather than the findings.

Stage 3 — Hardening: the critics attack the audit, not the paper

Hardening turns the machinery on the audit itself. Four critic roles attack the draft comment — statistics, data, fairness to the authors, and completeness — and the declared object of criticism is the audit, not the paper. Critics run from a different vendor and model family than the auditors. The current gate set fixes the defaults — the coordinator on Anthropic's Opus 5; the five auditors on Sonnet 5 at maximum reasoning effort; critics and synthesis on OpenAI's GPT 5.6 Terra, also at maximum effort — on the view that the cost of a missed finding dwarfs the cost of thinking harder. Defaults are not evidence, so the release notes still record model identity per role per round, and a post-hoc check reads the critic sessions' own logs to confirm which model actually served each one: a plan that silently downgrades on quota would otherwise attribute one model's findings to another. Critics review the current draft, not one keyed to a round number. Verdicts are CONFIRMED, REFUTED, OVERSTATED, or UNVERIFIABLE.

Every criticism is verified before it is applied. Wrong criticisms are contested, with the reproduction shown. Retired phrasings go to a retired-claims file so that no withdrawn claim reappears elsewhere in the document. A prompt gate blocks a round unless the prompt renders, carries no unsubstituted placeholder, and mentions a keyword from the paper under audit.

One structural fact governs how many rounds to run: text written in response to round r has not been reviewed by round r. Figure 2 shows the measured size of that blind spot — on the three comments measured so far, the text written after the last review was 43, 28, and 8 percent of the final word count. With a single round, nearly half a comment can ship unexamined, and it is exactly the text where the worst errors have appeared, because it is the newest.

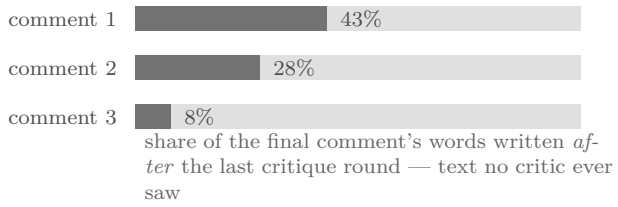


Figure 2. Why the second round exists. The bars measure each comment's unreviewed surface after one critique round: the words added or rewritten in response to round 1, which round 1 could not have read.

The production default is therefore **two rounds**: round 1 attacks the draft, round 2 exists to review round 1's own corrections, scoped by a wrap-up note naming the sections that changed. Contested or high-stakes audits raise the count toward a five-round cap. From round two the critic set shrinks from four roles to two, and the consolidating synthesis pass is dropped, because the failure mode moves: in round one the risk is a missed finding, which redundancy catches; from round two it is a wrong correction in the audit's own newest text, which two critics catch as reliably as four. Two audits conducted while this specification was being written support the threshold: in each, round one refuted a draft claim and added a finding the auditors had missed, while round two produced no new findings about the paper and only corrections to the audit's own record. Round one runs its four critics concurrently; the pre-flight probe catches a dead account or a rejected model before any critic starts, and each critic is individually resumable, so a throttled critic costs a retry rather than the round. Stopping — at the default, at convergence (a round returning no factual corrections, only wording), or on the cap — is recorded as which of the three it was, and the release notes state which sections no critic reviewed.

What the stage caught when the coordinator thought the comment was finished. On the two-arm comparison described above, every conclusion-changing claim had been re-derived by the coordinator before any critic saw the draft. The round still refuted three claims and narrowed twelve, in roughly forty minutes of critic time. The most consequential catch was a coordinator error, not an auditor one: verifying an auditor's count of reproduced derived values, the coordinator rebuilt one of the paper's formulas with an input term fixed at the cap the Methods specify, rather than applying the cap conditionally. The cap binds on every row of one condition and on only some rows of the other, so the simplification reproduced the first condition exactly — which is why it survived the coordinator's own checks — and corrupted the second. The auditor's original count was right and the coordinator's correction of it was wrong.

Two rules follow, and both are now in the skill. A simplification of a published formula must be validated in every regime the data occupy, not only the one it was tested against; a cap, floor or branch that changes state across rows is a scope boundary in the same sense as a file or a section. And when a coordinator re-derives an auditor's number and disagrees, the disagreement is a finding about one of the two and it is not yet known which: the resolution is a second, structurally different derivation, not the coordinator's authority. This is the same principle Stage 2 applies to disagreements between auditors, extended to disagreements between an auditor and the

coordinator.

Stage 4 — Release: the only stage that can refuse

Release is gated by two scripts carrying nine and twelve blocking checks, seven of them self-tests of the detectors they run (Appendix B). Both scripts deliberately omit shell abort-on-error, so every gate runs and failures accumulate; any accumulated failure exits nonzero and blocks release.

Handoff: the findings become someone else's checklist

The audit's last product is not the comment — it is the package that lets a human verifier confirm or refute every claim in it without trusting the machinery that produced them. Three artifacts share one join key, the claim number, and Figure 3 shows the chain.

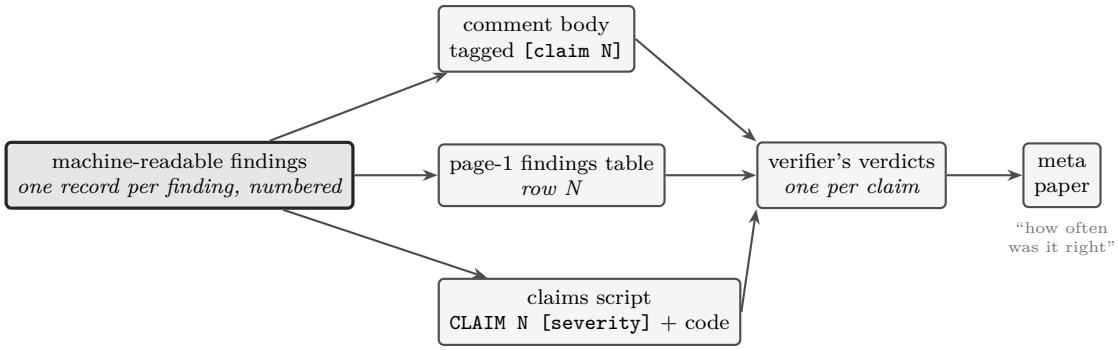
Every finding statement in the comment carries its number inline, and the same number heads a row of the appendix findings table. The published deliverable is built with the tags stripped; the working copy keeps them. A release gate blocks when table rows and body tags disagree, and its extractors validate themselves on planted forms before they scan anything. A separate check blocks a numbered findings table anywhere in the body: a comment is a paper, and a reader should meet the findings as prose before meeting them as an inventory.

The **claims script** is the executable half. One numbered block per finding, each carrying a severity label mapped mechanically from the finding's recorded impact and confidence — *retraction material* for a confirmed conclusion-level failure, *corrigendum material* for the value-level and likely cases, *comment material* for the rest, and nothing speculative ever rises above comment. Where the deposit ships code, the block carries the authors' own code verbatim, with file and line range, and the script itself takes the deposit's language and extension — a verifier who confirms audit-written code has confirmed the audit, not the paper. Audit-written checks stay in R: inline where the file is R, embedded as marked run-in-R blocks where it is not, and the whole file falls back to R when the deposit ships no code at all. Findings verified by inspection or arithmetic — an impossible p-value, a dataset that does not exist — go in a claims-with-no-code tail with a one-line manual check each, because they are most of the findings in the worst papers and they must receive verdicts like the rest.

The script's header carries the verifier's two standing rules. A retraction- or corrigendum-level claim the verifier cannot confirm after honest study means the paper goes back, not the claim: skipping it would let the machinery decide. And no verifier contacts the paper's authors or its journal; the coordinating institute handles all correspondence.

Because verdicts attach to numbered claims, they aggregate: each verified audit yields one row per claim — right, wrong, partially, cannot judge — and those rows are the raw material of the empirical agenda in Section 5. Finished audits are filed one folder per paper, holding exactly the delivered PDF and the claims script, so the state of a cohort is legible at a glance.

Figure 3. The handoff chain. The numbered findings record generates three tagged artifacts; the human verifier's per-claim verdicts join back on the claim number, and the joined table is what the later paper counts.



the claim number is the join key: never renumber one artifact without the others

The rule underneath: a zero proves nothing until the detector has fired

One rule underpins the design. A detector that returns zero has established nothing until it has fired on a known-present instance of every form it claims to cover, with its scope stated in advance on five axes: files, sections, lines, pattern forms, and text encodings. A clean result is a claim about coverage, and coverage must be demonstrated.

Two corollaries follow. Positive controls must exercise the code path and the target form used in production. And the protocol prefers completion signals a safeguard cannot forge — an exit code over a file's contents, a verdict marker over a file's existence, a regenerated artifact over an edited one. Absence of a signal is failure, not a pass.

Safeguards also compose badly while each is individually correct. Redaction protects participants and corrupts the derived tables the analysis needs. Telling critics to write output early prevents loss from cut-offs and produces files that look finished and are not. Appendix C lists twenty-two mechanisms by which a clean result can be wrong.

Disclosure: obligations that begin before any finding is public

Audits inspect data that may identify people, including minors, and they generate new linkage files of their own. Four rules apply. The package copy is read-only and the working tree stays off shared storage. Scans cover the artifacts the audit generates as well as those it inherits, because an intermediate file can itself carry participant data. Authors and the journal are notified privately before any public claim about a live exposure. And remedies verified analytically inert are preferred, with remediation verified rather than assumed.

4. Implementation

The protocol is one specification file, six reference documents, fourteen scripts, and six prompt templates. The reference documents are the auditor checklist, the shared-brief template, the adjudication guide, the comment's required section structure, the verification-notebook guide with a worked example, and the finding-classification scheme. The scripts create the read-only workspace, drive the critique rounds, freeze settled facts, run the consistency gate, stage the release, build the deliverable, generate the verification notebook and the claims script, record the machine-readable run and comment records, meter wall-clock and token cost per phase, confirm post hoc which model served each critic session, and file the finished audit into the completed-reports tree.

Synthesis is retained only where it earns its cost. Consolidating four critic reports into one dedupli-

cates and adjudicates between them; consolidating two does not, and the pass is a serial re-reading of work already done that measured 22 to 25 minutes per round. It is therefore skipped at two or fewer critics, and the coordinator reads the reports directly.

Each audit ships a fixed artifact set: the comment; the findings ledger; the adjudication log; the retired-claims file; the claims script and its claim-to-code map; a verification notebook that re-derives every checked number and refuses to knit on a failure; machine-readable run and comment records validated against published schemas; per-round critic outputs and consolidated critiques; driver logs and per-phase cost metadata; and release notes recording gate-set version, model identity per role per round, execution environment, rounds run, and whether stopping was at the production default, by convergence, or on the cap. Every quantity in Section 5 is computable from this set. Phase durations are recorded only where they were measured: a phase opened and closed in the same second was stamped retroactively rather than timed, and the record stores a null with a note saying so, because a zero that looks measured is worse than an acknowledged gap. At publication the final PDF, the claims script, and both machine-readable records are copied into a per-paper folder in the completed-reports tree, hash-verified, and the publish is refused if the pair has not been validated and filed by the finalisation step; the registry copy, keyed by audit identifier, remains canonical. The set is archived immutably at release, because records held in a session-scoped directory cannot be recovered once the session ends.

Two of the scripts are the release gates. The consistency gate carries nine blocking checks: the disclosure detector's own self-test; participant identifiers beside dates or clock times; retired claims surviving anywhere in the document, matched after normalising case, punctuation, and articles so a rephrased withdrawal cannot slip through; the executive summary's word limit; the presence of both page-one summary tables; the mandated citations; process narration; and two-way agreement between the findings table and the body's claim tags. The staging gate carries twelve, six of them self-tests, and delegates to a package-specific disclosure gate where one exists. Appendix B enumerates both.

The current gate set is **G-2026-08-21**, and it exists partly because its predecessor could not be cited. The original specification carried no version tag and no repository history; its only identifier was a file state, designated G-2026-07-30 after the fact — a defect in a document meant to be cited, since a reader of a comment could not otherwise tell which gate set produced it. Comments now name their gate set.

The false-clean discipline of Section 3, applied to our own gates, is what the succession records. G-2026-07-30 shipped with a live instance of mechanism M2 in Appendix C: a word-boundary anchor cannot fire on a token preceded by an underscore, which is itself a word character, so the disclosure detector returned clean on realistic identifier forms while firing on the positive control the script shipped — a control validated against the wrong form certifies a blind detector. **G-2026-08-21 repairs it** with explicit lookaround anchors, and its positive controls now include the underscore-led form that defeated the original. The predecessor's four silent-skip paths (mechanism M8) are narrowed rather than eliminated: a skipped check now announces itself in the gate output instead of printing success, but an announced skip still does not block, and mistaking one for a pass remains a reader error the gate permits. We record that residual rather than claim it away.

5. Empirical Agenda

This section freezes the definitions a later paper will use, so that paper is pre-committed rather than a search over outcomes. Every quantity is computable from the Section 4 artifacts and states

its unit of observation. Two are the headline quantities of Section 1: protocol compliance (Q15) and protocol reproducibility (Q16).

#	Quantity	Operational definition and unit	Estimation
Q1	Conclusion-changing share	Impact field records a change to an abstract claim, a numbered conclusion, or the final discussion paragraph. Unit: paper	Count, Wilson interval
Q2	Reproducible-but-wrong share	Headline quantities, fixed before reproduction, recovered to printed precision, with Q1 satisfied. Unit: paper	Two-by-two against reproduction; Fisher exact
Q3	Defect taxonomy shares	Primary category per finding, codebook frozen before the cohort; source is the ledger. Unit: finding, clustered by paper	Cluster-robust bootstrap intervals
Q4	One-line cause share	Most severe finding attributable to one lexical token in code or data, coded by two raters. Unit: paper	Proportion with sensitivity range
Q5	One-line remedy share	That token changed and unchanged code re-run suffices: no new estimand, uncertainty computation, or author decision. Unit: paper	Reported with Q4, never merged
Q6	Self-correction rate per round	S(r) = comment-defects among blocking items in round r; blocking means REFUTED, OVERSTATED, or listed as missed. Unit: round	By round and pooled
Q7	Contested-criticism rate	Criticisms contested rather than applied, and the share later shown correct. Unit: criticism	Proportions by round; the second is the error rate in self-defense
Q8	Coordinator-overruled rate	Adjudication entries marked WITHDRAWN, and the subset forced by an external critic. Unit: entry	Proportion, forced subset separately
Q9	False-clean incidence	Clean results later shown wrong, classified on the Appendix C list, each with a unique event identifier. Denominator: every "we found no X" statement. Unit: statement	Incidence per statement; control coverage per axis
Q10	Brief-correction rate	Auditor corrections to the intake brief, tiered CENTRAL, MATERIAL, INCIDENTAL. Unit: audit	Share with a CENTRAL correction, incentive disclosed
Q11	Rounds to convergence	Smallest r returning no factual corrections and only wording changes. Unit: audit	Share converged versus capped; sections no critic reviewed
Q12	Negative-result density	Hypotheses tested and refuted per comment, from the mandated section. Unit: comment	Ratio to substantive findings
Q13	Disclosure-defect incidence and lag	Live exposures identified; remedies verified inert; days from notification to verified remediation. Unit: audit	First two retrospective; third prospective

#	Quantity	Operational definition and unit	Estimation
Q14	Archive-defect incidence	Binary vector: no data; no code; does not execute; no environment record; broken pointer; no license. Unit: package	Six proportions
Q15	Protocol compliance	Rounds run; whether the delivered file postdates and incorporates the final critique; whether the gate ran; gate-set version; model identity per role per round; whether the record was archived. Unit: audit	Vector reported beside every estimate; non-compliant audits never pooled
Q16	Protocol reproducibility	Findings recovered when an independent team runs the identical protocol on the same package, judged by a blinded third party. Unit: finding, clustered by package	Recovery proportion, cluster-robust interval
Q17	Cost	Wall-clock time and tokens per stage and round, critic invocations, retries, monetary cost. Unit: round	Prospective only
Q18	Model-tier sensitivity	Findings recovered when the fan-out is re-run on the same package with a different auditor model, prompts and coordinator held fixed. Reported separately for findings the two tiers share, and for those recovered by one tier only, split by area and by whether the finding changes a conclusion. Unit: finding, clustered by package	Recovery proportion by tier and area; cluster-robust interval

Q18 was added after a controlled two-arm comparison showed the tier effect is concentrated in specific areas rather than uniform, which makes it a measurable design parameter rather than a procurement question. One quantity changed status without changing definition: Q17 was prospective-only at freeze, and the current gate set instruments it live — per-phase wall-clock and token metering writes an event log from which the cost table is derived, never hand-edited.

Five commitments accompany the list. The defect codebook is frozen before the cohort, not induced from it. The sampling frame is pre-registered, and every drawn package is dispositioned, including those that cannot be executed. The unit of analysis is fixed per quantity. Severity is coded by raters who did not generate the findings. Every artifact a quantity depends on is archived at release.

6. Scope and Limitations

This paper specifies the protocol and does not yet measure it. There is no baseline, no ablation, no human comparison arm, no independent re-execution, no ground truth for defects the protocol missed, and no quantity in Section 5 estimated. Nothing here establishes an error rate in either direction, or throughput and cost against human auditing. The components arranged in Section 3 are prior art in kind; the contribution is their arrangement, its written form, and the agenda that will test it.

Three threats survive into the later paper. Coordinator, auditors, and in practice some critics may share a model family, so their agreement is weak evidence; the partial mitigations are cross-family critics and the rule that disputed computations are re-run. Severity coding is currently done by the

party that generated the findings, which biases it in an unknown direction; Section 5 commits to independent raters. And auditors are told in advance that correcting the brief counts as a finding, so any brief-correction rate is confounded by its own incentive and must carry that disclosure.

A written protocol is not a followed protocol. That is why Q15 exists, and why non-compliant audits are reported separately rather than pooled.

7. Conclusion

The protocol arranges ordinary quality-assurance practice around one task: recomputing a published paper's statistics from its deposited package, with the audit's own output as the primary object of suspicion. Its distinguishing rules are few. The intake brief is fallible. Disagreements are settled by re-running rather than by preferring a source. Nothing is dropped without a recorded reason. The adversarial rounds attack the audit. And no detector is believed until it has fired on a known positive. Whether it works is an empirical question, and Section 5 states in advance the quantities that will answer it.

Competing interests

All authors are affiliated with the Institute for Replication, the organization that developed the protocol specified here, and the first author is the coordinator who executes it. Two of the works cited are the authors' own.

References

Aczel, B., Szaszi, B., and Holcombe, A. O. (2021). A billion-dollar donation: estimating the cost of researchers' time spent on peer review. *Research Integrity and Peer Review*, 6, 14. https://doi.org/10.1186/s41073-021-00118-2

Anaya, J. (2016). The GRIMMER test: A method for testing the validity of reported measures of variability. *PeerJ Preprints*, 4, e2400v1. https://doi.org/10.7287/peerj.preprints.2400v1

Bornmann, L., Haunschild, R., and Mutz, R. (2021). Growth rates of modern science: a latent piecewise growth curve approach to model publication numbers from established and new literature databases. *Humanities and Social Sciences Communications*, 8, 224. https://doi.org/10.1057/s41599-021-00903-w

Brodeur, A., Mikola, D., Cook, N., et al. (2026a). Reproducibility and robustness of economics and political science research. *Nature*, 652, 151–156. https://doi.org/10.1038/s41586-026-10251-x

Brodeur, A., Valenta, D., Marcoci, A., Aparicio, J. P., Mikola, D., Barbarioli, B., Alexander, R., Deer, L., Stafford, T., Vilhuber, L., Bensch, G., et al. (2026b). AI-assisted teams outperform AI-led teams but not human-only teams in assessing research reproducibility in quantitative social science. *PNAS*, 123(22), e2524747123. https://doi.org/10.1073/pnas.2524747123. Correction: *PNAS*, 123(28), e2621051123. https://doi.org/10.1073/pnas.2621051123

Brown, N. J. L., and Heathers, J. A. J. (2017). The GRIM Test: A Simple Technique Detects Numerous Anomalies in the Reporting of Results in Psychology. *Social Psychological and Personality Science*, 8(4), 363–369. https://doi.org/10.1177/1948550616673876

Camerer, C. F., Dreber, A., Forsell, E., et al. (2016). Evaluating replicability of laboratory experiments in economics. *Science*, 351(6280), 1433–1436. https://doi.org/10.1126/science.aaf0918

Carlisle, J. B. (2017). Data fabrication and other reasons for non-random sampling in 5087 randomised, controlled trials in anaesthetic and general medical journals. *Anaesthesia*, 72(8), 944–952. https://doi.org/10.1111/anae.13938

Chang, A. C., and Li, P. (2022). Is Economics Research Replicable? Sixty Published Papers From Thirteen Journals Say "Often Not". *Critical Finance Review*, 11(1), 185–206. https://doi.org/10.1561/104.00000053

Hardwicke, T. E., Mathur, M. B., MacDonald, K., et al. (2018). Data availability, reusability, and analytic reproducibility: evaluating the impact of a mandatory open data policy at the journal Cognition. *Royal Society Open Science*, 5(8), 180448. https://doi.org/10.1098/rsos.180448

Heathers, J. A. J., Anaya, J., van der Zee, T., and Brown, N. J. L. (2018). Recovering data from summary statistics: Sample Parameter Reconstruction via Iterative TEchniques (SPRITE). *PeerJ Preprints*, 6, e26968v1. https://doi.org/10.7287/peerj.preprints.26968v1

Herbert, S., Kingi, H., Stanchi, F., and Vilhuber, L. (2024). Reproduce to validate: A comprehensive study on the reproducibility of economics research. *Canadian Journal of Economics*, 57(3), 961–988. https://doi.org/10.1111/caje.12728

Institute for Replication (2026). Reproducing with AI Across the Expertise Ladder. OSF. https://osf.io/dkfzt

Kapoor, S., and Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9), 100804. https://doi.org/10.1016/j.patter.2023.100804

Liang, W., Zhang, Y., Cao, H., Wang, B., Ding, D. Y., et al. (2024). Can Large Language Models Provide Useful Feedback on Research Papers? A Large-Scale Empirical Analysis. *NEJM AI*, 1(8). https://doi.org/10.1056/AIoa2400196

Nuijten, M. B., Hartgerink, C. H. J., van Assen, M. A. L. M., Epskamp, S., and Wicherts, J. M. (2016). The prevalence of statistical reporting errors in psychology (1985–2013). *Behavior Research Methods*, 48(4), 1205–1226. https://doi.org/10.3758/s13428-015-0664-2

Nuijten, M. B., and Wicherts, J. M. (2024). Implementing statcheck During Peer Review Is Related to a Steep Decline in Statistical-Reporting Inconsistencies. *Advances in Methods and Practices in Psychological Science*, 7(2). https://doi.org/10.1177/25152459241258945

Open Science Collaboration (2015). Estimating the reproducibility of psychological science. *Science*, 349(6251), aac4716. https://doi.org/10.1126/science.aac4716

Siegel, Z. S., Kapoor, S., Nadgir, N., Stroebl, B., and Narayanan, A. (2025). CORE-Bench: Fostering the Credibility of Published Research Through a Computational Reproducibility Agent Benchmark. *Transactions on Machine Learning Research*. Preprint: arXiv:2409.11363.

Thelwall, M. (2024). Can ChatGPT evaluate research quality? *Journal of Data and Information Science*, 9(2). https://doi.org/10.2478/jdis-2024-0013

Vilhuber, L. (2020). Reproducibility and Replicability in Economics. *Harvard Data Science Review*, 2(4). https://doi.org/10.1162/99608f92.4f6b9e67

Vilhuber, L. (2023). Report of the AEA Data Editor. *AEA Papers and Proceedings*, 113, 850–863. https://doi.org/10.1257/pandp.113.850

Reference verification note. Every entry above was checked against a publisher record, Crossref, or a repository landing page. Two carry unresolved metadata, disclosed rather than left silent. A publisher correction to Brodeur et al. (2026a) exists, but its identifier was not retrieved, so the correction is not cited; the correction to Brodeur et al. (2026b) was retrieved and is cited alongside the article. The OSF identifier listed under Institute for Replication (2026) resolves to a live project whose public metadata returns a different working title, with no registered identifier and no components, so title, year, and authorship are unconfirmed. It is listed only as a pointer, because the release gate mandates the citation in every comment, and no claim in this paper rests on it.

Appendix A. The auditor checklist

Comments cite these items by number. Items 1 to 7 are distilled from published I4R comments; the remainder were added from calibration runs. The specification calls this an eleven-item checklist while the file carries twelve numbered items plus cross-cutting requirements; we record the discrepancy rather than resolve it silently.

1. **Duplication and sample inflation.** Exact duplicates; near-duplicates; contiguous block duplication at every offset; repeat submissions sharing a timestamp or contact; whole-sample duplication. Quantify the coincidence and name the mechanism. No near-duplicates in a survey export is not no near-duplicates in the response data.
2. **Impossible and out-of-range values.** Responses outside the scale; subscale scores above the arithmetic maximum; impossible demographics. Trace whether bad values entered published statistics.
3. **Reproduction and source discrepancies.** Sample size in data versus paper; means that reproduce while dispersions do not; text versus figure versus data; whole studies missing from an archive claiming completeness. Reproduce at the precision printed and enumerate every value that does not.
4. **Scoring and coding errors.** Subscale formulas mixing opposing-construct items; inconsistent composition across studies; wrong multipliers; silent recoding; scale-offset bugs that manufacture a group difference. Verify the column-to-item mapping against the covariance structure before judging an intended key.
5. **Statistical-reporting anomalies, no data required.** Algebraically impossible statistics; sign errors; missing decimals; mislabeled captions; a wrong direction in an abstract; inconsistently labeled confidence levels.
6. **Signatures of fabricated or random data.** Absent latent structure; uniform distributions with sharp cut-offs where skew is expected; impossibly narrow ranges; absurd effect sizes. Benchmark against published reference datasets.
7. **Privacy and archive hygiene.** Personal data in nominally anonymized files, and whether an anonymization claim was actually made. Compute uniqueness per person, not per identifier value. Check filenames and embedded document metadata, not only cells.
8. **Estimator specification and library defaults.** Which function is called, and which arguments are left at defaults. Print the returned object and check that the element being read is the intended one. Vary the defaults; check release notes of pinned dependencies for default changes.

9. **Figure fidelity.** Measure plotted values against the text. Axis limits that remove observations differ from coordinate zooms; a clipped panel silently drops data from its own caption's sample size.
10. **Mechanical reproducibility.** Does the shipped code run as shipped? Undeclared dependencies; placeholder paths; runtime fetches by hard-coded identifier; locale-dependent parsing; absent environment capture; a path that silently loads the wrong study's data.
11. **Treatment fidelity, not only outcome provenance.** When a component is cleared as not the source of the dependent variable, ask separately whether it is part of the manipulation.
12. **Reporting omissions.** Computed but unreported analyses. Report them as omissions, not errors, and attribute no motive.

Cross-cutting requirements. Label every claim CONFIRMED, LIKELY, or SPECULATIVE. List benign explanations considered and how each was ruled out. State impact on published conclusions for every finding; most have none. Report refuted hypotheses as prominently as confirmed ones. Credit what the authors did well, specifically. Verify the remedy, not only the defect: every finding ending "the authors should do X" asserts that X is possible and sufficient.

Appendix B. The release gates

The consistency gate carries fourteen blocking checks. A self-test of the disclosure detector aborts the run if either planted probe fails to match — including the underscore-led identifier form that defeated the previous gate set. Three content checks look for a participant identifier on the same line as a date, the same beside a clock time, and any retired claim appearing anywhere in the document, matched after normalising case, punctuation, and articles. Six format checks follow: the comment must carry an Abstract heading; the abstract must not exceed 250 words; it must be prose, with no table inside it; no numbered findings table may appear anywhere before the appendix; no Recommendations section may exist, because each finding carries its own remedy beside its evidence; and both mandated citations must be present. A further check blocks process narration. The last three cross-check the appendix findings table against the body's claim tags in both directions — a table row never tagged in the body, or a tag with no row, blocks release — and validate their two extractors on planted forms before scanning anything. An advisory word-budget line estimates standing against the page cap; the binding page check runs on the built document itself, as does a check that the abstract finishes on the first page.

The staging gate carries twelve blocking checks, six of them self-tests of the detectors it runs: code-detector probes, a timestamp probe, a negative probe asserting that ordinary prose does not fire, a density probe, and an absolute-path probe in both directions. The remaining six check that the staged tree contains the required artifacts, that the deliverable was regenerated rather than edited, that no absolute path from the auditing machine survives in the deliverable, that the gate-set version is recorded, that the release notes name model family and identity per role per round, and that a package-specific disclosure gate ran where one exists.

Both scripts deliberately omit shell abort-on-error, so every check runs and a failure flag accumulates. Any accumulated failure exits nonzero and blocks release. Section 4 records the defect repaired between G-2026-07-30 and G-2026-08-21, and the residual it left.

Appendix C. Twenty-two false-clean mechanisms

Mechanisms only: the entries are failure modes to check for, carrying no incidence counts. They are the recognizable ones from software-quality practice — inadequate input coverage, boundary

and encoding defects, fail-open behavior, tests exercising the wrong path, stale fixtures, and self-certification — assembled in the form a reproducibility audit needs at hand. Entries are named generically so that a comment can cite a mechanism by number.

#	Mechanism	Illustration
M1	The pattern cannot express the target's format	A birth-year pattern run against full dates; an identifier pattern that assumes one naming convention
M2	A word-boundary anchor never fires	A `\b` anchor before a token preceded by an underscore, which is itself a word character
M3	Encoding scope wrong	UTF-16 strings invisible to an 8-bit pattern; a byte-order mark silently renaming a column; document formats never decoded although they ship
M4	File scope wrong	A scan covering one subdirectory while the claim covered the staged tree; a suffix allow-list that omits the data formats
M5	Section scope wrong	A retired-claims check that reads the abstract and not the body
M6	A conjunction too strict	A gate requiring an identifier and a timestamp on the same line, when real rows put a measurement between them
M7	A gate prints FAIL and exits zero	The failure is visible in the log and the release proceeds; detectable only by planting a violation
M8	A silent skip reported as a pass	A missing claims file, an undelimitable abstract, an empty keyword, or an absent versioned draft, each treated as nothing to check
M9	A positive control validates the wrong code path	The control exercises one matcher while the detector uses another
M10	A positive control validated against the wrong form	Planted year-bearing dates certify a detector blind to day-month dates
M11	A positive control trips the gate it validates	Mitigated by splitting literals and using out-of-range values; the mitigation itself can become dead code
M12	A superseded revision reviewed or staged	A critique round reviews a draft already revised; a stale report is staged silently
M13	A stale prompt produces confident, irrelevant output	A critic reviews a different paper entirely; the failure surfaces as reassurance, not as an error
M14	A detector run outside its preconditions	Granularity tests on non-integer inputs; a duplicate-detection null destroyed by legitimate block missingness
M15	A detector with no power	A dispersion screen that accepts nearly the whole candidate space at the relevant subgroup sizes; an underpowered digit test
M16	Vacuous input	A scan over empty rows; a correction keyed to a participant code that does not exist; a range check on a column coerced to text
M17	A wrong denominator	Re-identification risk divided by distinct key combinations rather than by released records

#	Mechanism	Illustration
M18	An arithmetic reconciliation that closes for the wrong reason	A total that balances under two different explanations, only one of which was checked
M19	A seed that does not pin the result	A fixed split seed operating on a hash-ordered set; resampling with no seed call at all
M20	A guard that masks the defect on the production path	A length guard writes missing values only on full-length runs, so printed tables look clean while saved predictions are corrupt
M21	Verifying the wrong dimension	Every cell verified against its cited source line while the error is in the column heading
M22	The integrity mechanism certifies itself	A ledger self-audit that tracks only findings already in the draft, and so cannot detect a finding that never reached it

Several of these mechanisms are created by the audit rather than inherited from the paper under audit, because safeguards compose badly (Section 3). The design response is to prefer completion signals that a safeguard cannot forge, and to treat the absence of a signal as failure rather than as a pass.